23. Metrics & Monitoring System

> **Interviewer:** Design a metrics and monitoring platform — think Datadog or Prometheus. Thousands of hosts and services need to emit numbers like CPU usage, request latency, and error counts. We need to store all of it so engineers can graph it on dashboards, and we need to **alert** automatically when something crosses a threshold, like "page the on-call if error rate > 5% for 5 minutes."

Candidate: Good one — this is fundamentally a **numeric time-series** problem: enormous write volume of tiny points, versus much rarer but latency-sensitive reads for dashboards and alerts. I want to flag up front that this is distinct from a *logging* system — logs are unstructured text blobs you search; metrics are structured numbers you aggregate and graph. Let me clarify scope before I design.

---

1. Clarifying the requirements

> **Candidate:** A few questions: 1. What's actually being sent — just system metrics (CPU, memory), or also application/business metrics (request latency per endpoint, orders placed)? 2. Is collection **push** (agents send data to us) or **pull** (we scrape agents on a schedule), or do I get to choose? 3. How far back do people need full-resolution data — is "zoom into the last 5 minutes from 6 months ago" a real requirement, or just "recent"? 4. For alerting — what's an acceptable false-page rate vs. missed-page rate? Which way do we err?
>
> **Interviewer:** Both system and application metrics, and let's allow arbitrary custom metrics from services too. You can choose the collection model, just justify it. Full resolution only needs to be available for a couple of days; older data can be coarser. On alerting: **a missed alert is much worse than a duplicate or slightly-delayed one** — this is how people find out production is broken.

Candidate: Great, that pins down the design goals. Let me write requirements.

Functional - Agents/services emit metrics: `name + tags + timestamp + value` (e.g. `http.latency{host=srv-17, endpoint=/checkout} = 212ms @ t`). - Store every point as part of a time-series, queryable by metric name + tag filters + time range. - Dashboards can query and graph any series (or aggregate across many series) over a range. - Users define **alert rules** ("if `avg(error_rate) > 5%` over 5m, notify on-call") evaluated continuously against live data.

Non-functional — this is where it's interesting - **Extreme write volume, small payloads.** Every host, every few seconds, forever. Writes outnumber reads by orders of magnitude. - **Cardinality explosion risk.** The *number of distinct time-series* (not just the number of points) can blow up if tags aren't bounded — this is the metrics-specific failure mode, different from a raw throughput problem. - **Fast recent-data reads.** Dashboards refresh every few seconds; "last hour" must return in well under a second, even while ingest is at full throttle. - **Storage must stay bounded** even though data never stops arriving — can't keep everything at full resolution forever. - **Alerts: never silently miss one.** Some duplication or a few seconds of delay is fine; silence during an outage is not.

2. Estimation — and the cardinality trap

> **Candidate:** Let me size the raw throughput first, then the thing that actually breaks naive designs: cardinality.

```
10,000 hosts, each emitting 50 metrics, once every 10 seconds:
  points/sec = 10,000 * 50 / 10 = 50,000 points/sec  (steady state)
  peak during incidents/deploys (more metrics, more retries) ~= 150,000/sec

Each point ~ 50-100 bytes (metric name + tags + timestamp + value):
  50,000/s * 100B = ~5 MB/s = ~430 GB/day of RAW data
  -> clearly cannot keep this at full resolution forever. That's WHY
     downsampling (section 6) has to exist, not a nice-to-have.

Now the trap: CARDINALITY.
  A "series" = one unique combination of metric name + tag values.
  If tags are bounded (host, region, endpoint) -> maybe 10,000 hosts *
  20 endpoints * 5 regions ~= 1M series. Totally manageable in a TSDB index.

  But if a well-meaning engineer tags a metric with `user_id` or
  `request_id` (each unique per request):
    50,000 points/sec, EACH one is potentially a brand-new series
    -> the series index itself grows unboundedly, at ~the SAME rate
       as raw writes. The index (which must fit in memory to be fast)
       explodes and the whole TSDB grinds to a halt or OOMs.

  This is the metrics-specific failure mode: it's not "too much DATA,"
  it's "too many DISTINCT SERIES." A system that handles 50k points/sec
  fine can be brought to its knees by ONE badly-tagged metric.
```

Candidate: So there are really two numbers I have to design around: **raw point throughput** (a storage/ingest problem, solvable with the usual scaling tools) and **series cardinality** (a design/guardrail problem — I need bounded-cardinality tags plus limits that reject or quarantine runaway metrics before they take down the index).

---

3. A simple V1

> **Candidate:** Let me state a naive version, mostly to show what's *right* about it (the data model) before I scale the parts that break.

```
  [ Agent on host ] --POST /ingest {metric, tags, ts, value}--> [ Server ] --> [ Postgres ]

    Table: points(series_id, ts, value)
    Query: SELECT ts, value FROM points
           WHERE series_id = ? AND ts BETWEEN ? AND ?
```

Why this breaks quickly: at 50,000 writes/sec, that's 50,000 B-tree-index maintenance operations/sec, forever, on a table growing to billions of rows — that ceiling is reached fast. Queries are always "a **time**

range for one series," which a generic row-store index isn't shaped for (no notion of "only touch the blocks covering this hour"). And nothing here downsamples, so storage grows unboundedly at ~430 GB/day.

Candidate: The data model concept (metric name + tags identify a series; each point is `(timestamp, value)` in that series) is the right idea — I just need a store built around *appending to a sorted-by-time series* instead of a generic table.

---

4. The time-series data model, precisely

> **Interviewer:** Let's nail the data model before the storage engine — what exactly is a "series," and how do you key it?

Candidate:

```
One data point:
  metric:    "http.request.latency"
  tags:      { host: "srv-17", region: "us-east", endpoint: "/checkout" }
  timestamp: 1737100000
  value:     212.5      (a float — always a NUMBER, that's what makes this "metrics"
                         not "logs")

SERIES = metric name + the exact set of tag key/values, hashed to a series_id.
  series_id = hash("http.request.latency", {host:srv-17, region:us-east, endpoint:/checkout})

Conceptually, a wide-column / LSM layout:
-------------------------------------------------------------------------
 series_id     (hash of metric+tags)         PARTITION KEY  — routes to a shard
 time_bucket   (which hour/day block)        CLUSTERING KEY — picks the file/block
 timestamp     (exact second)                CLUSTERING KEY — sort order within block
 value         (float)
-------------------------------------------------------------------------

Query "cpu.usage, host=srv-17, last 6h" ->
  1. resolve tag filters -> series_id(s)   (an inverted index: tag -> series_ids)
  2. for each series_id, scan only the 6 hourly blocks in range
  3. merge + return sorted points
```

The inverted index is the other half of the picture: to answer "give me *every* series where `region=us-east`" (for a dashboard aggregating across a whole region), I need a secondary index from `tag_key=tag_value -> [series_ids]`, exactly like a search engine's inverted index. This index is also exactly what blows up under high cardinality — it's the "series catalog," and it must fit in memory to keep queries fast.

---

5. Storage engine — the TSDB, and scaling ingest

> **Interviewer:** OK, walk me through the actual storage engine, and how it survives 150,000 writes/sec.

Candidate: A purpose-built **Time-Series Database** (Prometheus's TSDB, InfluxDB, M3DB, or a wide-column store like Cassandra used in this pattern), fronted by a queue so ingest spikes never hit it directly.

```
                  INGEST / WRITE PATH
[ 10,000 agents ] --batched HTTP POST (100s of points/req)--> [ L7 LB ]
     |  routes /ingest across many stateless collectors
     ▼
[ Collectors ]  (validate shape, reject obviously-bad cardinality, cheap)
     ▼
[ Kafka: metrics topic, partitioned by series_id ]   <- absorbs bursts
     ▼
[ Aggregation / write workers ]  (each owns a slice of series_ids)
     ├─► append raw points to TSDB, per-series, time-ordered
     |       (in-memory write buffer -> immutable compressed block
     |        on disk every ~2m, like an LSM tree)
     └─► also feed the downsampling pipeline (section 6)
     ▼
[ Time-Series Database, sharded by series_id ]
```

Why append-only + block-per-time-window is the whole trick: - We are **always writing the newest point** for a series — never updating an old row. That's the cheapest possible write: append to an in-memory buffer, no B-tree rebalancing. - Data is **partitioned by time window** — one immutable block per, say, 2 hours. A query for "the last 6 hours" touches ~3 blocks, not the whole dataset. This is the single biggest reason a TSDB beats a general-purpose DB here. - **Compression is specific to this shape of data:** timestamps are almost evenly spaced (delta-of-delta encoding gets this to a couple of bits per point), and values like CPU rarely jump wildly (XOR/delta encoding). Real TSDBs get **10-20x compression** over a naive row store — this matters enormously at 430 GB/day of raw input. - **Sharding by `series_id`** (not by time) spreads the 150,000 writes/sec across many nodes — no single hot shard, since writes are naturally spread across thousands of independent hosts/series to begin with (metrics ingest is "embarrassingly parallel," *as long as cardinality stays bounded*).

---

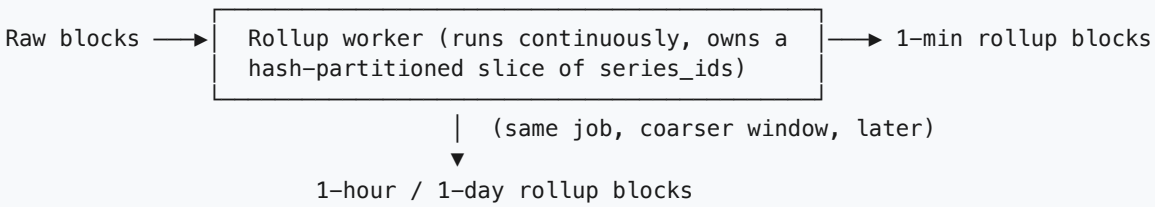
6. Downsampling & rollups — keeping storage bounded

> **Interviewer:** 430 GB/day forever is not "bounded." What do you actually keep?

Candidate: This is the async, off-the-hot-path piece: **rollup workers** continuously compute coarser aggregates from raw data and age out the raw data behind them.

```
RAW (10s resolution)  ──[after 2 days]──▶  1-MINUTE rollups  ──[after 2 weeks]──▶  1-HOUR ro
kept ~2 days                kept ~2-4 weeks                       kept ~1-2 years

A rollup point isn't just an average — store enough to answer real queries later:
  rollup = { count, sum, min, max, sum_of_squares (for stddev) }
so "avg over this hour" = sum/count, and we don't silently lose min/max spikes
by only storing a mean.
```



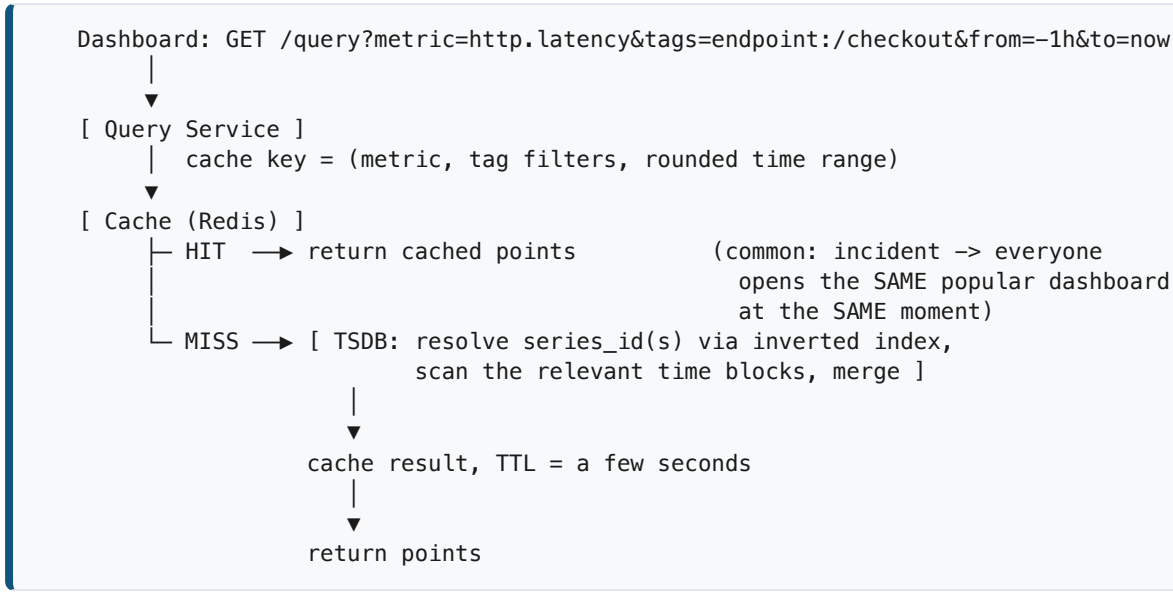
- This is exactly like a photo getting blurrier the further back you go — recent data is crisp (10s resolution) for live debugging; month-old data is blurry (hourly) because nobody needs 10-second precision on an incident from six months ago, and keeping it would cost 100x more.
- **Percentiles are the sharp edge here:** you cannot reconstruct an accurate p99 later from only a rolled-up *average*. If p99 latency matters (it usually does), the rollup must store a pre-aggregated histogram/sketch (e.g., a t-digest or HDRHistogram summary) at write time, not just min/max/avg — average-only rollups quietly destroy the ability to ever answer "what was our p99 during that incident."
- Query layer is rollup-aware: "last hour" reads raw blocks; "last 6 months" transparently reads hourly-rollup blocks. The user just sees a graph — they don't know which tier answered it.

---

7. Query path & caching

> **Interviewer:** A dashboard asks for "checkout latency, last hour" during an active incident, and 500 engineers open the same dashboard. How do you keep that fast?

Candidate: Cache-aside, but with a **short TTL** — this data is genuinely different from, say, a URL shortener's cache, because "the last hour" keeps changing every few seconds.



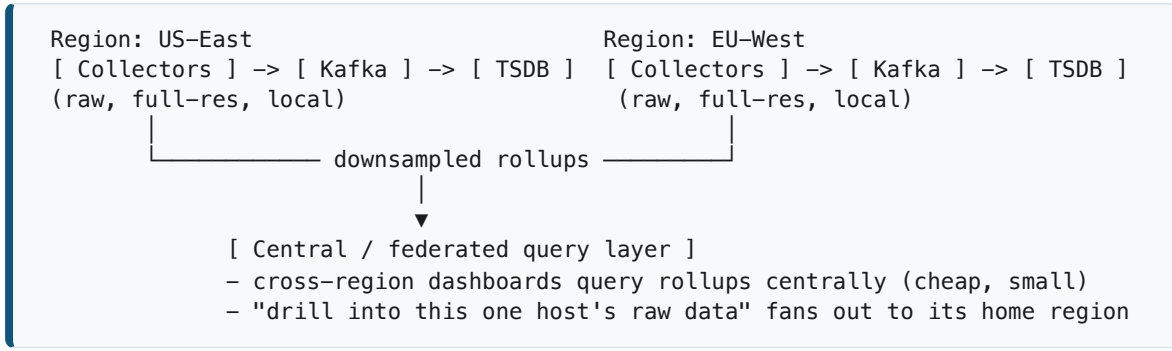
- **Why cache at all if the TTL is only a few seconds?** Because the *value* isn't from long-lived reuse — it's from **fan-in**: during an incident, hundreds of people load the identical query within the same few seconds. The cache collapses that fan-in into one TSDB read instead of hundreds.
- **Why not a long TTL like a typical read-heavy cache?** A stale "last hour" graph during an active incident is actively harmful — it can hide the very spike people are trying to see. Short TTL trades a little redundant TSDB load for correctness on live data.

---

8. Multi-region

> **Interviewer:** We're a global company — hosts in US-East and EU-West. How does that change things?

Candidate: Raw data should stay close to where it's produced; only *compact* data should travel far — same principle as the estimation section, applied geographically.



Why not ship every raw point cross-region? At 5 MB/s per region, shipping full-resolution data across an ocean link for every host, all the time, is wasted bandwidth on a pipeline already at the edge of what it can absorb. Two valid patterns, same tradeoff as always: **centralize** rollups for "one global view" dashboards, and **federate** (scatter-gather) queries for "give me the raw detail on this host" — fan out only when someone needs it.

---

9. Consistency — strong where it must be, approximate everywhere else

> **Interviewer:** What's actually strongly consistent here, and what's allowed to be fuzzy?

Candidate: Almost everything here is fine to be **eventually consistent / approximate** — that's actually the whole design philosophy of a monitoring system, and it's worth saying out loud because it's the opposite instinct from, say, a payments system.

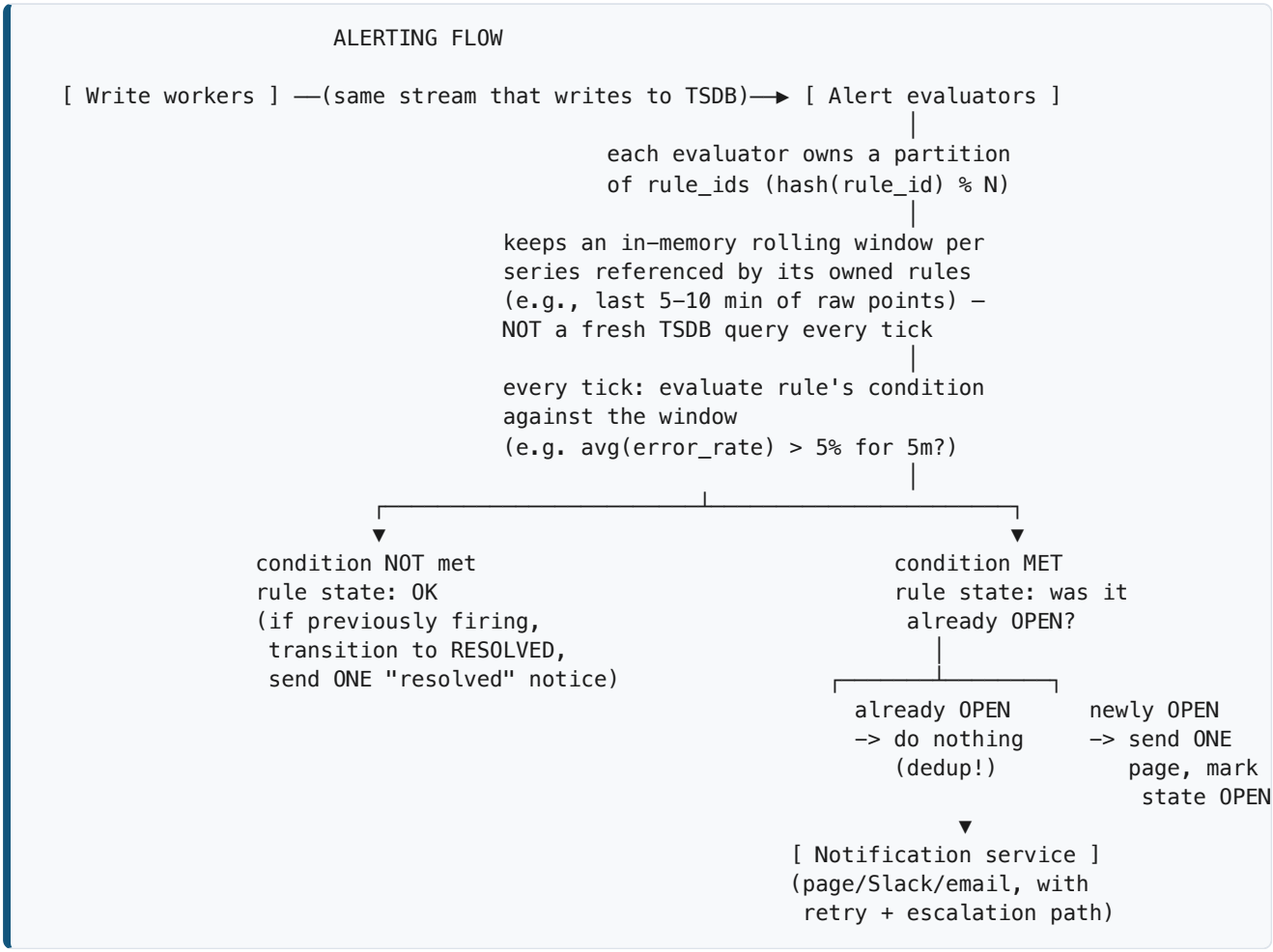
- **Eventual, and that's fine:** a point written at `t` might not be queryable for a second or two while it moves through Kafka and the write worker. A dashboard showing "as of 3 seconds ago" during a failover is completely acceptable — nobody needs the literal current millisecond, they need the trend.
- **Approximate, by design:** rollups are *lossy* on purpose (that's the point of downsampling); percentile sketches (t-digest/HDRHistogram) are approximations of the true p99, not exact values, and that's an accepted tradeoff for being able to compute them cheaply at write time.
- **The one place I do want strong guarantees: alert *state***. "Is this rule currently breaching, yes or no" must be evaluated against a single, agreed-upon view — not two evaluator replicas independently deciding different things from slightly different data, because that's how you get either a missed page or a duplicate one. I'll make one evaluator instance own each rule (partition rules by `rule_id`, like partitioning series by `series_id`), so there's a single source of truth for "did this rule just flip."

---

10. The alerting engine

> **Interviewer:** Now design the alerting piece specifically — thousands of rules, each needs continuous evaluation against a live stream, and a missed alert is the worst outcome.

Candidate: Alerting is a **continuous query engine**, not a one-off lookup — I don't want each rule re-scanning the full TSDB on every tick; I want a streaming evaluator with a small warm window of recent data per rule.



- **Dedup via explicit state, not "don't evaluate twice."** The evaluator ticks every cycle regardless — that's what makes it robust to crashes. What we dedup is the *notification*: only send a page on the state **transition** (`OK -> OPEN` or `OPEN -> RESOLVED`), never on every tick a rule continues breaching. That's how "check every 30s for 5 minutes straight" doesn't become 10 pages for one incident.
- **Rule ownership by partition** prevents the double-page version of the same problem: two evaluator replicas both deciding "this just opened" for the same rule at the same moment. One owner per rule at a time removes that race — the same "atomic claim" idea used to avoid double-processing elsewhere, applied to a `rule_id`.
- **Crash recovery: at-least-once, never at-most-once.** If an evaluator dies mid-cycle, its rules get reassigned (consistent hashing, like Kafka consumer-group rebalancing) and re-evaluated fresh — no reliance on a fragile in-memory "already sent" flag a crash could wipe. Worst case: one duplicate notification (already deduped by state); never a silently missed breach — the correct bias per section 1.

---

11. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Time-series storage	**Purpose-built TSDB** (Prometheus TSDB / InfluxDB	Append-only writes (always the newest point, no in-place updates), time-based block partitioning (a "last 6h" query only touches recent blocks), and

Concern	Choice	Why this, and why not the alternative
	/ M3DB, or Cassandra in a time-series schema)	specialized compression (delta-of-delta on timestamps, XOR/delta on values) that a generic store doesn't do.
Why not a relational DB (Postgres/MySQL)	rejected	A B-tree index has to rebalance on every one of 50,000-150,000 inserts/sec — that ceiling is reached fast. Range-over-time queries also aren't what a general row-store index is shaped for; you'd be fighting the engine's natural access pattern the whole way.
Why not a plain KV store	rejected	Great for single-key point lookups, but "every point for this series in this time range" is a *range* scan, which most KV stores don't optimize for at billions-of-keys scale. A TSDB is essentially a KV store specialized exactly for range-over-time.
Ingest buffering	**Kafka**, partitioned by `series_id`	Durable, ordered buffer that absorbs bursts (a deploy or incident spiking metric volume) so write workers drain at a steady pace instead of the TSDB taking the raw spike directly; partitioning by series preserves per-series ordering and gives natural sharding for downstream workers.
Downsampled/rollup storage	Same TSDB, coarser time-partitioned blocks (1-min -> 1-hour -> 1-day tiers)	Reuses the same storage engine and query path; the query layer just picks the tier appropriate to the requested range. Keeps storage ~100x smaller for old data while still answering "what did last month look like on average."
Percentile support	Pre-aggregated histograms/sketches (t-digest, HDRHistogram) computed **at write time**	You cannot accurately reconstruct p99 later from only stored averages — the information is already gone by then. Must be captured before rollup discards raw points.
Recent-query cache	**Redis, short TTL (seconds)**	Absorbs the "everyone opens the same incident dashboard at once" fan-in; short TTL because "last hour" data changes every few seconds and a stale incident graph is actively misleading.
Series index (tag -> series_id)	In-memory inverted index inside the TSDB, with cardinality guardrails	Needed for "all series where region=us-east"-style queries; must stay small enough to live in memory, which is exactly why unbounded tags (user_id, request_id) are rejected/quarantined at the collector rather than allowed to explode the index.
Alert evaluation	Dedicated streaming evaluator service, partitioned by `rule_id`, in-memory rolling window per rule	Avoids re-querying the full TSDB per rule per tick; partitioning gives one owner per rule so two replicas can't double-page; state-transition-based notification (not "was it true this tick") gives natural dedup.
Collection model	**Push** (agents send data out) as the default, with	Push works simply behind firewalls/NAT and needs no central registry of every target; the tradeoff (discussed in follow-ups) is pull makes "who

Concern	Choice	Why this, and why not the alternative
	pull/scrape (Prometheus-style) as a supported option	stopped reporting" trivially visible as a failed scrape.

The one-sentence justification you'd say out loud: "*A time-series database with time-based partitioning and specialized compression is the only fit for append-only writes queried by time range at this volume; Kafka in front of it absorbs ingest bursts so the TSDB never sees a raw spike directly; downsampling keeps storage bounded by trading resolution for age, with percentile sketches captured before that lossy step happens; a short-TTL cache absorbs the thundering herd of identical dashboard queries during an incident; and the alerting engine is a partitioned streaming evaluator that dedupes on state transitions, biased firmly toward a duplicate page over a missed one.*"

---

12. Failure modes & graceful degradation

Failure	What happens	Mitigation
Ingest spike (deploy, incident triggers more metrics)	Write workers can't keep up in real time	Kafka absorbs the burst; workers drain at their own steady pace — a few seconds of ingest lag, never data loss
Cardinality explosion (bad tag, e.g. `request_id`)	Series index grows unboundedly, TSDB memory pressure, everything slows	Collector-level guardrails: cap distinct tag values per metric, reject/quarantine metrics that exceed a series-count budget, alert the offending team
Agent/host goes offline	That series simply stops receiving points — a gap in the graph	The monitoring system alerts on its own absence of data ("no heartbeat from host X in 5m") — a missing signal is itself a signal
TSDB node dies	Its shard of series is briefly unavailable	Replication (N=3 typical) — replicas keep serving reads/writes for that shard; failed node rebuilds from replicas on recovery
Alert evaluator crashes mid-cycle	Its owned rules stop being checked	Rules reassigned to remaining evaluators (consistent hashing / rebalance) and re-evaluated fresh against current data — at-least-once, accept a possible duplicate over a silent gap
Same breach evaluated twice (retry, rebalance overlap)	Risk of double-paging	State-based dedup: only notify on `OK->OPEN` / `OPEN->RESOLVED` transitions, not on every tick a rule remains breaching
Notification channel (pager) itself down	Alert fires but nobody's reached	Retry with backoff + a secondary escalation channel — an alert nobody receives is as bad as one never fired

---

13. Follow-up curveballs

> **Interviewer:** Push or pull collection — which would you actually pick, and why?

Candidate: I lean **push** as the default for a heterogeneous fleet: agents just send data out, which works trivially behind NAT/firewalls and needs no central registry of every host that exists. The real tradeoff is **pull** (Prometheus-style scraping): the server hits each target's `/metrics` endpoint on a schedule, which makes "who stopped responding" trivially visible — a failed scrape *is* the down signal, no separate heartbeat needed — at the cost of the central scraper needing service discovery for every target. At large, dynamic fleets (containers churning constantly), push tends to scale operationally better; Prometheus's pull model shines in a more static, well-known target list.

> **Interviewer:** How do you keep alert evaluation fast when a rule needs the last hour of data, and you have 10,000 rules ticking every 30 seconds?

Candidate: Never re-query the full TSDB per rule per tick — that's 10,000 range scans every 30 seconds and would itself become a top read bottleneck. Instead, each evaluator keeps a small **in-memory rolling window** (just the lookback the rule actually needs — 5-60 min) per series it's responsible for, fed incrementally as new points arrive off the same stream that feeds the TSDB writers. The rule evaluates against that warm window in memory — O(window size), not O(dataset).

> **Interviewer:** Someone wants to write a dashboard query like "p99 latency across all `/checkout` instances, last 24h." Walk me through what that actually touches.

Candidate: Resolve the tag filter `endpoint=/checkout` through the inverted index to get the matching `series_id`s (one per host serving that endpoint) → for each, read the relevant rollup tier for a 24h range (probably 1-minute rollups, not raw) → since p99 doesn't average across series, the rollups need to carry a mergeable summary (a histogram/sketch, not just a scalar avg) so the query layer can merge N per-host histograms into one true cross-host p99 at read time. This is exactly why "store the sketch at write time" from section 6 matters — you can't fake a merged p99 out of pre-averaged numbers after the fact.

> **Interviewer:** A dashboard needs to correlate a latency spike with a deploy event. How does that fit in?

Candidate: That's an **annotation**, not a metric — a discrete event (deploy at time T, by whom, what version) rather than a continuous numeric series. I'd store deploy/incident events in a lightweight separate store (even a simple table keyed by time), and have the dashboard overlay annotation markers on the same time axis as the metric graph. It's a much lower-volume write path than metrics, so it doesn't need the TSDB's machinery — just a shared time axis at render time.

---

14. Why not just use a relational database?

> **Interviewer:** Postgres is battle-tested, has indexes, partitioning, and every engineer already knows it. Why not just make a `metrics` table — `(series_id, ts, value)` — and add a couple of indexes? Section 3 already tried this and moved on quickly; convince me it's actually the wrong call, not just an unfamiliar one.

Candidate: It's the right *first instinct* — the data model (series + timestamp + value) is correct, I said that in section 3. What's wrong is putting it in a general-purpose row store, for three compounding reasons: write volume, what a row actually costs, and what "recent data" queries need.

```
1. WRITE VOLUME vs. WHAT A B-TREE COSTS PER WRITE
   50,000-150,000 points/sec, forever.
   Every INSERT into an indexed table = find the right B-tree leaf,
   possibly split a page, write WAL, fsync (or batch-fsync) it.
   That's 50k-150k of those PER SECOND, not once - this is the
   steady state, not a burst. A B-tree is built for "occasional
   writes, frequent point lookups" - metrics ingest is the opposite
   shape: constant writes, range-over-time reads.

2. ROW-PER-DATAPOINT EXPLODES
   One row = series_id (8B) + timestamp (8B) + value (8B)
             + row header/MVCC overhead (~24B in Postgres, tuple
               header + visibility info) + index entry copy (~16-24B)
             ≈ 60-70 bytes of ACTUAL DISK for an 8-byte float.
   Compare: a TSDB stores that same float in ~1-2 bytes after
   delta-of-delta timestamp + XOR value encoding, because it knows
   every row is "the next point in an already-sorted series" and
   never needs MVCC/visibility bookkeeping for it.
      -> Postgres: 50k pts/s * 65B  ≈ 3.25 MB/s  ≈ 280 GB/day
      -> TSDB:     50k pts/s * 1.5B ≈ 75 KB/s    ≈  6.5 GB/day
   Same data, ~40x smaller, because the storage engine is shaped for
   the access pattern instead of fighting it.

3. THE QUERY SHAPE POSTGRES ISN'T BUILT FOR
   Every real query here is "one series (or a tag-filtered set of
   series), a time RANGE." A B-tree index on (series_id, ts) helps,
   but the underlying heap file still isn't physically laid out by
   time - Postgres has to consult the index then fetch scattered heap
   pages. A TSDB's blocks ARE the time partitioning: "last 6h" means
   "read these 3 files," full stop, no index indirection.
```

Concern	Relational DB (Postgres/MySQL)	Purpose-built TSDB (Prometheus / InfluxDB / M3DB)
Write path	B-tree insert per row: index maintenance, MVCC tuple overhead, WAL/fsync per write — tuned for occasional writes + point lookups	Append to an in-memory buffer, flush as an immutable time-ordered block (LSM-style) — tuned for constant sequential writes
On-disk size per point	~60-70 bytes (value + row header + MVCC bookkeeping + index entry)	~1-2 bytes (delta-of-delta timestamps, XOR/delta-encoded values)
"Last 6 hours" query	Index lookup, then fetch scattered heap pages — no guarantee they're physically adjacent	Time-partitioned blocks — the query touches only the 2-3 blocks covering that window, nothing else

Concern	Relational DB (Postgres/MySQL)	Purpose-built TSDB (Prometheus / InfluxDB / M3DB)
Downsampling / rollups	Bolted on: a cron job or materialized view you build and maintain yourself	Native concept — same storage engine, coarser time-partitioned tier, query layer already tier-aware
Cardinality (many series)	Millions of distinct `series_id`s just means a bigger table + bigger index — no inherent ceiling *from Postgres's side*, but nothing warns you either	Series count is a first-class metric the TSDB tracks and can guard/limit on — cardinality is a known failure mode with built-in tooling
Where it wins	Ad-hoc joins, transactional writes, strong relational integrity	None of that — metrics have no natural joins or transactions, so you're paying relational overhead for a feature you don't use

> **Rule of thumb:** if the write pattern is "always append the newest value, keyed by (series, time), read back by time range" — that's a time-series database's entire job description. Reaching for a relational DB there means paying for B-tree point-lookup machinery and MVCC bookkeeping on every single write, for a query pattern (time-range scan) the engine wasn't shaped for — and reinventing downsampling and compression yourself instead of getting them for free.

---

15. Hard edge case, walked through — the cardinality explosion

> **Interviewer:** Your inverted index and TSDB are sized for a bounded set of tags. A backend engineer adds one line — `metric.tag("user_id", user.id)` — to an existing counter, ships it, and goes home. Walk me through exactly what happens next, and how you'd have caught or prevented it.

Candidate: This is the failure mode I flagged back in section 2 — it's not "too much data," it's "too many *distinct series*," and the multiplier is brutal because it's exponential in the number of unbounded tags, not additive.

```
BEFORE: metric = http.requests.count
        tags   = { host (10,000 values), endpoint (~20 values) }
        series = 10,000 * 20 = 200,000 distinct series   <- fits in memory fine

ONE LINE CHANGES:
        tags = { host (10,000), endpoint (~20), user_id (~50,000 active users) }
        series = 10,000 * 20 * 50,000 = 10,000,000,000 (10 BILLION) possible
                 combinations in theory; in practice, bounded by actual
                 REQUEST traffic, but still:

        every (host, endpoint, user_id) combo that ever occurs becomes a
        BRAND NEW series the first time it's seen -> at 50,000 requests/sec
        with mostly-distinct user_ids in that window, you mint roughly
        50,000 NEW series PER SECOND, not 50,000 points into 200,000
        existing ones.

        10 series --[add user_id tag]--> 10 * (~1M distinct users) = ~10,000,000 series
        (10x hosts * endpoints, unchanged)      (the ONE unbounded tag dominates everything)

WHAT BREAKS, IN ORDER:
  1. Inverted index (tag -> series_id) is in-memory by design (section 4) -
     it now needs an entry per NEW series, growing at ~ingest rate instead
     of being basically static. Memory climbs without bound.
  2. Each series also carries its own small write buffer / chunk metadata
     in the TSDB -> per-series fixed overhead * 10,000,000 = the TSDB's
     resident memory footprint explodes even though total POINT volume
     barely changed.
  3. Compaction/rollup workers, which assumed "a knowable, mostly-stable
     set of series," now churn constantly creating rollup state for
     series that will NEVER be queried again (that user_id won't repeat).
  4. End state: TSDB nodes OOM or GC-thrash, write latency spikes, and
     because ingest and alert evaluation SHARE the same write stream
     (section 10), alert evaluators degrade too -> the monitoring system
     itself becomes the outage, at the worst possible time.
```

Ranked fixes:

1. **Label hygiene / cardinality limits at design time (best long-term fix).** Treat "which tags are allowed on a metric, and how many distinct values each may take" as part of the metric's schema, reviewed like an API contract — `user_id`, `request_id`, `session_id`, `email`, or anything with effectively unbounded distinct values simply never becomes a tag. Cheapest fix because it prevents the problem instead of reacting to it.
2. **Per-metric cardinality budgets enforced at the collector, with alerting.** Even with good intentions, mistakes ship — so the collector (section 5) tracks distinct series-per-metric and rejects or quarantines a metric once it crosses a budget (e.g. 10,000 series), paging the owning team instead of letting it silently degrade the shared index. This is the safety net under fix #1, not a replacement for it.
3. **Drop or aggregate the high-cardinality label at ingest.** If a team genuinely needs `user_id`-level detail for *debugging*, strip it before it reaches the metrics pipeline and instead emit an aggregate (`http.requests.count` without `user_id`) plus, separately, whatever per-user detail they need goes elsewhere (fix #4). A relabeling rule at the collector can do this centrally without waiting for every team to fix their code.
4. **Route high-cardinality data to logs or traces instead of metrics.** This is the architectural fix, not just a patch: "which specific user hit this error" is a *lookup* question over discrete events — exactly what logs (indexed by structured fields) and traces (indexed by trace/span ID) are built for.

Metrics are for aggregates over a bounded label space; the moment you want to filter down to an individual entity, you've actually described a logging or tracing query wearing a metrics costume.

5. **Quarantine and backfill-drop after the fact (reactive, last resort).** If it's already in production and paging you, the collector guardrail (#2) should already be rejecting new series for that metric; ops additionally deletes the already-created runaway series to reclaim index memory immediately, rather than waiting for TTL expiry.

Recommended approach: treat cardinality budgets as a *first-class part of the metrics schema*, enforced automatically at the collector (fix #2) as the safety net, with fix #1 (review what tags are allowed before code ships) as the real prevention — and route anything that's inherently per-entity (user, request, session) to logs/traces (fix #4) rather than ever trying to force it through a metrics pipeline. This mirrors the split from section 1: metrics are numbers you aggregate over a *bounded* dimension space; the moment a dimension is unbounded, it isn't a metric tag anymore, it's a query filter that belongs in a different system.

> **What to say out loud:** "*Cardinality, not raw point volume, is the failure mode that actually takes a metrics system down — one unbounded tag can turn 200,000 series into tens of millions because the multiplier is across every existing tag combination, not additive. I'd defend against it in layers: schema review before a tag ships, an automated per-metric cardinality budget enforced at the collector as the safety net, and a firm architectural line that per-entity data — anything keyed by user, request, or session — belongs in logs or traces, not metrics, because metrics are built for aggregation over a bounded label space, not lookup over an unbounded one.*"

Summary — the mental model

A metrics platform is **"absorb an enormous stream of tiny numeric points cheaply, store them so time-range reads are fast and storage stays bounded, and evaluate thousands of alert rules continuously without ever going silent."** The winning moves: a **time-series database** with time-partitioned, compressed, append-only blocks instead of a relational table; **Kafka** in front to absorb ingest bursts; **downsampling/rollups** (with percentile sketches captured *before* the lossy step) to keep storage bounded as data ages; a **short-TTL cache** to survive the "everyone opens the same dashboard during an incident" fan-in; and a **partitioned, state-transition-driven alert evaluator** that is deliberately biased toward one extra duplicate page rather than any risk of missing a real breach. The single design rule that cuts across all of it: **watch cardinality, not just volume** — a bounded number of well-tagged series scales gracefully; one unbounded tag can take the whole system down regardless of how well everything else is built.